Docker + MySQL

Skript und Handout zur Live-Session

Trainer: Sebastian Meisel | Team Viona Lernbegleitung

```
$ docker compose up -d  # MySQL lokal starten
```



Ziel der Session

Teilnehmende installieren Docker, starten MySQL im Container und nutzen eine Datenbank mit Beispieldaten.

1. Orientierung: Was wurde live gezeigt?

Die Live-Session zeigt den Weg von einer frischen Docker-Installation bis zu einem laufenden MySQL-Container. Danach wird erklärt, wie man die Datenbank praktisch nutzt: verbinden, Datenbank anlegen, Tabellen erstellen, Datensätze einfügen und abfragen.

Schritt 1

Docker-Repository einrichten

Paketquellen, Schlüsselring und Docker-Quelle werden eingerichtet. Dadurch installiert das System Docker aus dem offiziellen Docker-Repository.

```
sudo apt update
sudo apt install ca-certificates curl
```

Schritt 2

Docker Engine installieren

Installiert werden Docker Engine, CLI, containerd sowie die Compose-Erweiterung.

```
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose
```

Schritt 3

Compose-Datei erstellen

Die Datei docker-compose.yml beschreibt den MySQL-Dienst, Portfreigabe, Passwort und persistentes Volume.

```
mkdir MySQL
cd MySQL
nano docker-compose.yml
```

Schritt 4

Container starten und prüfen

Der Container läuft im Hintergrund. Mit docker ps sieht man, ob MySQL gestartet wurde.

```
docker compose up -d
sudo docker ps
```

Merksatz

Docker liefert die Laufzeitumgebung. docker-compose.yml beschreibt den gewünschten Zustand. MySQL speichert Daten im Volume, damit sie nach einem Neustart des Containers erhalten bleiben.

2. Sanitierter Chat-/Shell-Verlauf

Der Verlauf ist bereinigt und didaktisch geglättet. Er zeigt die Arbeitsfolge aus der Live-Session.

Live-Verlauf, bereinigt

```
sudo apt update

sudo apt install ca-certificates curl

sudo install -m 0755 -d /etc/apt/keyrings

sudo curl -fsSL https://download.docker.com/linux/debian/gpg \
  -o /etc/apt/keyrings/docker.asc

sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update

sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

mkdir MySQL
cd MySQL
cat << _ > docker-compose.yml
services:
  database:
    image: mysql:latest
    container_name: mysql
    environment:
      MYSQL_ROOT_PASSWORD: temp1234
    ports:
      - "3306:3306"
    volumes:
      - mysql_volume:/var/lib/mysql
volumes:
  mysql_volume:
_

sudo docker compose up -d
sudo docker ps
```

! **Hinweis zu Compose**

Nach Installation des Docker-Compose-Plugins lautet der moderne Befehl meistens docker compose mit Leerzeichen. docker-compose mit Bindestrich ist die ältere Schreibweise.

3. Die docker-compose.yml verstehen

docker-compose.yml

```
services:
  database:
    image: mysql:latest
    container_name: mysql
    environment:
      MYSQL_ROOT_PASSWORD: temp1234
    ports:
      - "3306:3306"
    volumes:
      - mysql_volume:/var/lib/mysql
volumes:
  mysql_volume:
```

services	Liste der Container-Dienste	ein Dienst: database
image	Container-Image	mysql:latest nutzt aktuelle MySQL-Version
container_name	fester Containername	praktisch für docker exec
environment	Variablen im Container	setzt hier Root-Passwort
ports	Portfreigabe Host:Container	localhost:3306 -> MySQL
volumes	dauerhafte Speicherung	Daten bleiben erhalten

! **Sicherheitsnotiz**

Das Passwort temp1234 ist nur für Übungssysteme geeignet. In echten Umgebungen: starkes Passwort, keine Klartext-Secrets im Repository, Zugriff auf Port 3306 begrenzen.

4. Datenbank nutzen: Verbindung und Beispieldaten

Sobald der Container läuft, kann man MySQL entweder direkt im Container oder mit einem Client wie MySQL Workbench, DBeaver oder der Konsole nutzen.

Verbinden

```
# Prüfen, ob der Container läuft
sudo docker ps

# MySQL-Konsole im Container öffnen
sudo docker exec -it mysql mysql -uroot -ptemp1234

# Alternative mit Client auf dem Host:
# Host: localhost
# Port: 3306
# Benutzer: root
# Passwort: temp1234
```

Beispieldaten anlegen und abfragen

```
CREATE DATABASE kursverwaltung;
USE kursverwaltung;

CREATE TABLE teilnehmende (
  id INT AUTO_INCREMENT PRIMARY KEY,
  vorname VARCHAR(50) NOT NULL,
  kurs VARCHAR(80) NOT NULL,
  status VARCHAR(20) NOT NULL,
  startdatum DATE NOT NULL
);

INSERT INTO teilnehmende (vorname, kurs, status, startdatum) VALUES
  ('Alex', 'Fachinformatik AE', 'aktiv', '2026-08-01'),
  ('Mina', 'Fachinformatik SI', 'aktiv', '2026-08-01'),
  ('Sam',  'IT-Grundlagen',     'pause', '2026-07-15');

SELECT * FROM teilnehmende;
SELECT kurs, COUNT(*) AS anzahl FROM teilnehmende GROUP BY kurs;
```

So erklären

Eine Datenbank ist der Container für Tabellen. Eine Tabelle hat Spalten wie id, vorname oder status. Ein Datensatz ist eine Zeile. Mit SELECT fragt man Daten ab, mit INSERT fügt man Daten hinzu.

5. Typische Arbeitsbefehle

Container anzeigen	sudo docker ps	zeigt laufende Container
Logs prüfen	sudo docker logs mysql	hilft bei Startproblemen
Container stoppen	sudo docker compose down	stoppt Container, Volume bleibt
Wieder starten	sudo docker compose up -d	startet anhand der Compose-Datei
MySQL-Konsole	sudo docker exec -it mysql mysql -uroot -ptemp1234	öffnet SQL-Konsole
Datenbanken zeigen	SHOW DATABASES;	SQL-Befehl in MySQL
Tabellen zeigen	SHOW TABLES;	zeigt Tabellen der aktiven DB

Kurzcheck

Ich kann erklären, wofür Docker bei MySQL genutzt wird.

Ich kann eine docker-compose.yml grob lesen.

Ich kann mich mit MySQL verbinden und eine Datenbank auswählen.

Ich kann einfache INSERT- und SELECT-Befehle ausführen.

6. Backup, Export und Import mit mysqldump

mysqldump wird genutzt, um Datenbanken oder einzelne Tabellen als SQL-Datei zu exportieren. Wichtig: mysqldump liest nicht ein. Der Import passiert mit dem MySQL-Client mysql. Die Dump-Datei selbst sieht aber so aus, als käme sie aus mysqldump.

mysqldump: Export und Import

```
# Backup aus dem laufenden Container erzeugen
sudo docker exec mysql mysqldump -uroot -ptemp1234 kursverwaltung > kursverwaltung_backup.sql

# Backup mit Tabellenstruktur und Daten
sudo docker exec mysql mysqldump -uroot -ptemp1234 --databases kursverwaltung > kursverwaltung_backup.sql

# Dump-Datei wieder einlesen
sudo docker exec -i mysql mysql -uroot -ptemp1234 < kursverwaltung_demo_dump.sql

# Danach prüfen
sudo docker exec -it mysql mysql -uroot -ptemp1234
SHOW DATABASES;
USE kursverwaltung_demo;
SHOW TABLES;
SELECT * FROM teilnehmende;
```

Praxis-Hinweis

Für den Unterricht ist die Import-Variante besonders praktisch: Alle Teilnehmenden starten mit denselben Tabellen und denselben Beispieldaten. Danach können SELECT, JOIN, GROUP BY, UPDATE und DELETE sicher geübt werden.

Beispielabfragen nach dem Import

```
USE kursverwaltung_demo;

-- Alle Teilnehmenden mit Kurs und Modul anzeigen
SELECT t.vorname, t.nachname, k.titel AS kurs, m.titel AS modul, e.status
FROM teilnehmende t
JOIN einschreibungen e ON e.teilnehmende_id = t.id
JOIN kurse k ON k.id = e.kurs_id
JOIN module m ON m.kurs_id = k.id
ORDER BY t.nachname, m.reihenfolge;

-- Wie viele aktive Teilnehmende gibt es pro Kurs?
SELECT k.titel, COUNT(*) AS aktiv
FROM kurse k
JOIN einschreibungen e ON e.kurs_id = k.id
WHERE e.status = 'aktiv'
GROUP BY k.titel;
```

7. Übungen und Notizen

Übungen

- Ändere den Container-Namen gedanklich: Welche Befehle müssten danach angepasst werden?
- Erstelle eine zweite Tabelle module mit id, titel und dauer_wochen.
- Füge drei Beispieldatensätze in module ein.
- Schreibe eine SELECT-Abfrage, die nur aktive Teilnehmende zeigt.
- Erkläre, warum das Volume mysql_volume wichtig ist.

Lernbrücke

Docker beantwortet die Frage: Wo läuft MySQL? SQL beantwortet die Frage: Wie arbeite ich mit den Daten? Beide Ebenen gehören zusammen, sind aber nicht dasselbe.

Eigene Notizen

Das folgende Feld ist ein ausfüllbares PDF-Formularfeld.